

Reinforcement Learning

LAB 5

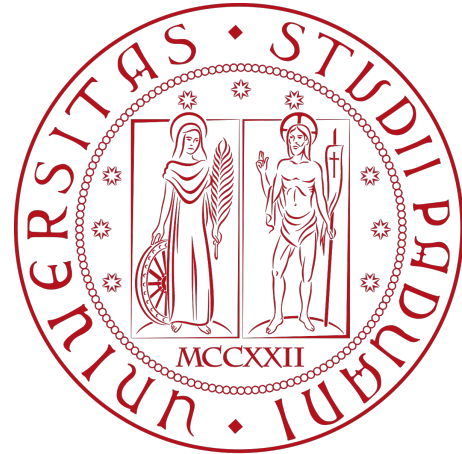
Temporal Difference

Riccardo De Monte
riccardo.demonte@phd.unipd.it

Alberto Sinigaglia
alberto.sinigaglia@phd.unipd.it



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE



Once upon a time, in very dark times...

Last lab we saw prediction and control using dynamic programming, exploiting the knowledge of the state transition matrix $\mathbf{P}$ of the MDP. However, it might be that such transition is unknown, or too messy to deal with.

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Long live Monte Carlo

Thus, we switched to Monte Carlo to estimate the Value function:

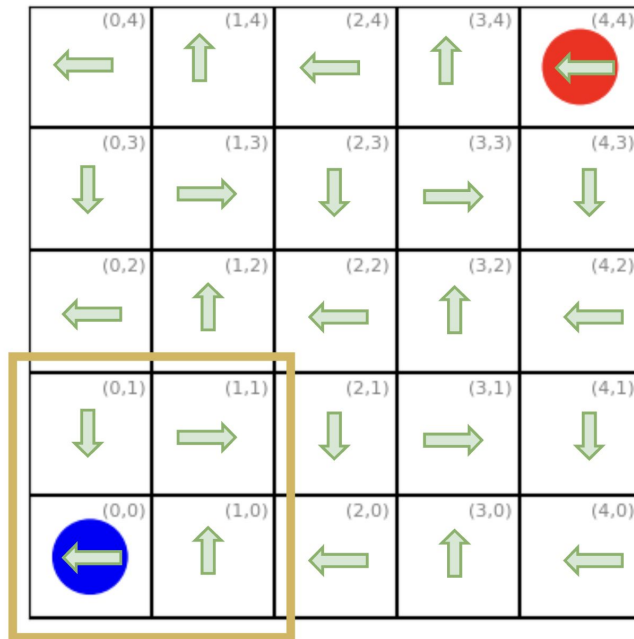
$$V(\boldsymbol{s}) = \mathbb{E}_{\tau \sim P(\tau | \boldsymbol{s}_t = \boldsymbol{s})} \left[\sum_{i=t}^T \gamma^{i-t} r_{i+1} \right]$$

Competition: develop the best policy

In particular, we left last time with this environment/MDP.

The goal of the competition was to develop the best policy possible, and we so that to evaluate a policy we have to consider the following quantity:

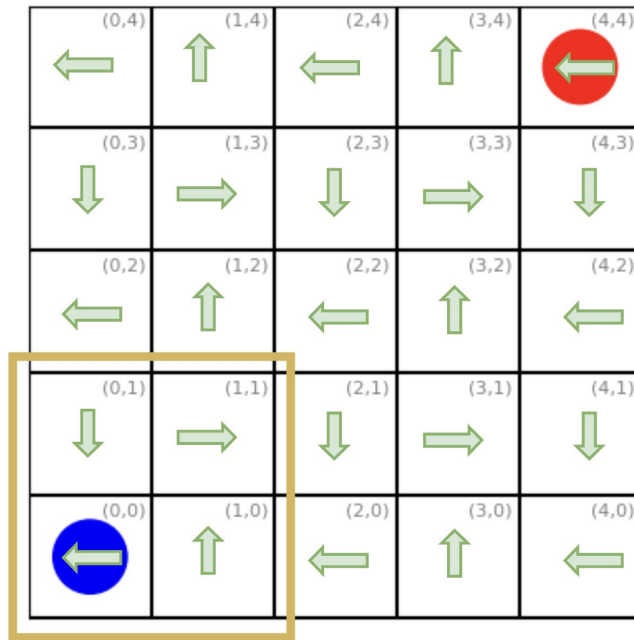
$$\mathbb{E}_{s_0 \sim \mu_0}(\cdot) [V(s_0)]$$



Competition: develop the best policy

However, for the ones that have tried,
developing a policy from scratch is not really
“easy”, and the fact that RL gives us a way to find
how good it is, is not really something amazing:

Ideally, we want RL to gives us the best policy



Recall the Action-Value Function

Sutton's definition for the action-value function:

$$Q(s, a) = \mathbb{E} \left[\sum_{i=t}^T \gamma^{i-t} r_{i+1} \middle| s_t = s, a_t = a \right]$$

Recall the Action-Value Function

Sutton's definition for the action-value function:

$$Q(s, a) = \mathbb{E} \left[\sum_{i=t}^T \gamma^{i-t} r_{i+1} \middle| s_t = s, a_t = a \right]$$

“Alberto’s notation” for the state-value function:

$$V(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [\mathbb{E}_{s' \sim p(\cdot|s,a)} [\mathbb{E}_{r \sim p(\cdot|s,a,s')} [r + \gamma V(s')]]]$$

Recall the Action-Value Function

Sutton's definition for the action-value function:

$$Q(s, a) = \mathbb{E} \left[\sum_{i=t}^T \gamma^{i-t} r_{i+1} \middle| s_t = s, a_t = a \right] \quad Q(s, a) = \mathbb{E}[G_t | s_t = s, a_t = a]$$

“Alberto’s notation” for the state-value function:

$$V(s) = \mathbb{E}_{a \sim \pi(\cdot | s)} [\mathbb{E}_{s' \sim p(\cdot | s, a)} [\mathbb{E}_{r \sim p(\cdot | s, a, s')} [r + \gamma V(s')]]]$$

Therefore:

$$V(s) = \mathbb{E}_{a \sim \pi(\cdot | s)} [Q(s, a)]$$

Monte Carlo Control

Therefore , for any pair (s,a) , the return (discounted cumulative reward) is an UNBIASED estimate of $Q(s,a)$ for the current policy.

Any problem?

Monte Carlo Control

Therefore , for any pair (s,a) , the return (discounted cumulative reward) is an UNBIASED estimate of $Q(s,a)$ for the current policy.

1. No online updates
2. It works for episodic tasks
3. High Variance (bad for control)

TD instead of MC

$$\begin{aligned} Q(s, a) &= \mathbb{E}_{s' \sim p(\cdot|s, a), r \sim p(\cdot|s, a, s'), \tau \sim P(\cdot|s_{t+1}=s')} [r + \gamma G_{t+1}] \\ &= \mathbb{E}_{s' \sim p(\cdot|s, a), r \sim p(\cdot|s, a, s')} [r + \gamma V(s')] \\ &= \mathbb{E}_{s' \sim p(\cdot|s, a), r \sim p(\cdot|s, a, s')} [r + \gamma \mathbb{E}_{a' \sim \pi(\cdot|s')} [Q(s', a')]] \end{aligned}$$

RL is an approximation of Dynamic Programming (prediction)

We cannot compute the actual expectations:

1. Dynamic Programming provides ideal update rules.

RL is an approximation of Dynamic Programming (prediction)

We cannot compute the actual expectations:

1. Dynamic Programming provides ideal update rules.
2. Instead, for all (s,a) we sample from the MDP (r,s') .

RL is an approximation of Dynamic Programming (prediction)

We cannot compute the actual expectations:

1. Dynamic Programming provides ideal update rules.
2. Instead, for all (s,a) we sample from the MDP (r,s') .
3. We approximate the Dynamic Programming update rules with Monte Carlo Estimation:

$$Q(s, a) \approx \frac{\sum_{s', r, G_{t+1}} (r + \gamma G_{t+1})}{N}$$

$$Q(s, a) \approx \frac{\sum_{s', r} (r + \gamma \mathbb{E}_{a' \sim \pi(\cdot|s')} [Q(s', a')])}{N}$$

SARSA or Expected SARSA??? (Prediction)

SARSA update has higher variance than Expected SARSA, but it does not require an exact expected value w.r.t. the policy (complicated for continuous action space):

$$Q(s, a) \approx \frac{\sum_{s', r} (r + \gamma \mathbb{E}_{a' \sim \pi(\cdot | s')} [Q(s', a')])}{N} \quad \text{Expected SARSA}$$

$$Q(s, a) \approx \frac{\sum_{s', r, a' \sim \pi} (r + \gamma [Q(s', a')])}{N} \quad \text{SARSA}$$

What about control?

We do policy evaluation estimating Q and then use an epsilon-greedy policy.

How to perform policy evaluation for control?

What about control?

We do policy evaluation estimating Q and then use an epsilon-greedy policy.

How to perform policy evaluation for control?

First option (the code for this lab relies on this):

$$Q(s, a) \approx \frac{\sum_{s', r} (r + \gamma \mathbb{E}_{a' \sim \pi(\cdot | s')} [Q(s', a')])}{N} \quad \text{Expected SARSA}$$

$$Q(s, a) \approx \frac{\sum_{s', r, a' \sim \pi} (r + \gamma [Q(s', a')])}{N} \quad \text{SARSA}$$

What about control?

We do policy evaluation estimating Q and then use an epsilon-greedy policy.

How to perform policy evaluation for control?

Second option (example with SARSA):

$$Q(s, a) = Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a))$$

Why constant step size for control???

Q-learning and off-policy control

With Expected SARSA, we estimate Q for the current policy

$$Q(s, a) = \mathbb{E}_{s' \sim p(s'|s, a), r \sim p(\cdot|s, a, s')} [r + \gamma \mathbb{E}_{a' \sim \pi(\cdot|s')} [Q(s', a')]]$$

While Q-learning approximates:

$$Q^*(s, a) = \mathbb{E}_{s' \sim p(s'|s, a), r \sim p(\cdot|s, a, s')} [r + \gamma \max_{a'} Q^*(s', a')]$$

Off-policy control

1. Consider any sampling policy $b(a|s)$.
2. Since a' is the only random quantity that depends on the policy:

$$\mathbb{E}_{s' \sim p(s'|s,a), r \sim p(\cdot|s,a,s')} \left[r + \gamma \mathbb{E}_{a' \sim b(\cdot|s')} \left[\frac{\pi(a'|s')}{b(a'|s')} Q_{\pi}(s', a') \right] \right]$$

Off-policy control

Q-learning is off-policy:

1. Consider any sampling policy $b(a|s)$.
2. Since a' is the only random quantity that depends on the policy:

$$\mathbb{E}_{s' \sim p(s'|s,a), r \sim p(\cdot|s,a,s')} \left[r + \gamma \mathbb{E}_{a' \sim b(\cdot|s')} \left[\frac{\pi(a'|s')}{b(a'|s')} Q_{\pi}(s', a') \right] \right]$$

3. If our target policy is greedy:

$$\frac{\pi(a'|s')}{b(a'|s')} = 1(a' = \operatorname{argmax}_{\bar{a}} Q_{\pi}(s', \bar{a})) \cdot \frac{1}{b(a'|s')}$$

Q-learning and off-policy control

1. Recall that Q-learning is approximating:

$$Q^*(s, a) = \mathbb{E}_{s' \sim p(s'|s, a), r \sim p(\cdot|s, a, s')} [r + \gamma \max_{a'} Q^*(s', a')]$$

Q-learning and off-policy control

1. Recall that Q-learning is approximating:

$$Q^*(s, a) = \mathbb{E}_{s' \sim p(s'|s, a), r \sim p(\cdot|s, a, s')} [r + \gamma \max_{a'} Q^*(s', a')]$$

2. Q^* is the one for the optimal policy:

$$\pi(a|s) = \pi(s) = \operatorname{argmax}_a Q^*(s, a)$$

Q-learning and off-policy control

1. Recall that Q-learning is approximating:

$$Q^*(s, a) = \mathbb{E}_{s' \sim p(s'|s, a), r \sim p(\cdot|s, a, s')} [r + \gamma \max_{a'} Q^*(s', a')]$$

2. Q^* is the one for the optimal policy:

$$\pi(a|s) = \pi(s) = \operatorname{argmax}_a Q^*(s, a)$$

3. Therefore:

$$\mathbb{E}_{a' \sim \pi(s)} [Q^*(s', a')] = \max_{a'} Q^*(s', a')$$

Final Remarks

1. SARSA and Expected SARSA do not converge to the optimal policy (we have to add GLIE exploration).
2. Q-Learning instead converges given ANY behaviour policy b (the policy should ensure exploration of all pairs (s,a)).
3. Q-Learning does not have any policy improvement step: we directly learn Q^* .
4. Epsilon-greedy is used for practicality: it ensures both exploration and exploitation.